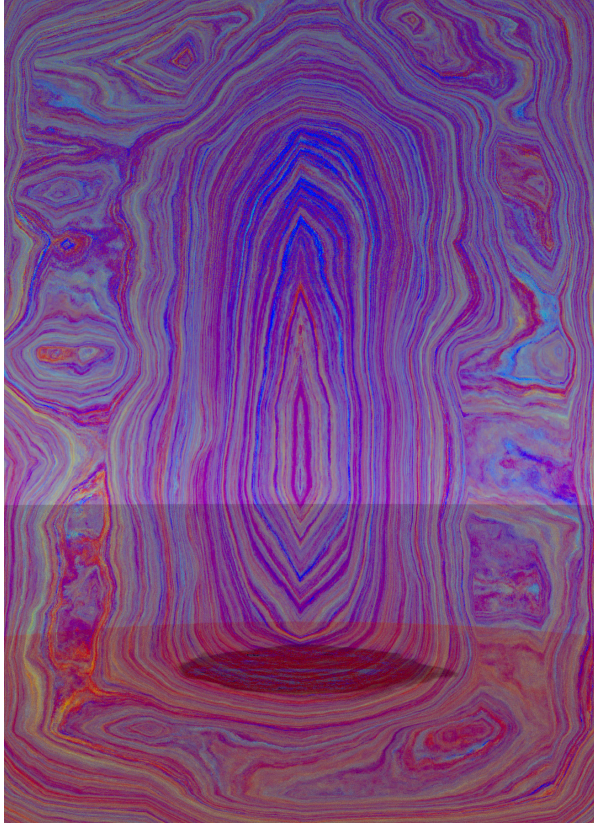
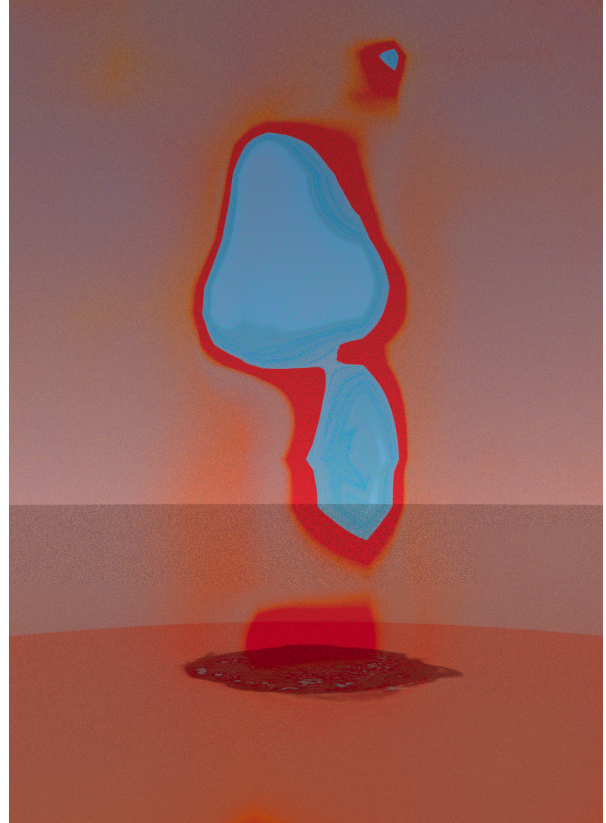


## Radiance and Throughput Slice mismatch

During the evaluation of the results, after the integration, there a significant issue when attempting to visualize the radiance. This was the case even when only utilizing the red channel for throughput on smaller networks with less noisy slices, which was that radiance was not being seen where there was supposed to be fire being emitted. Below is an example:



large\_color\_view1.png

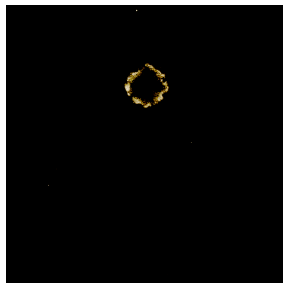


medium\_tAfter\_linear\_color.png

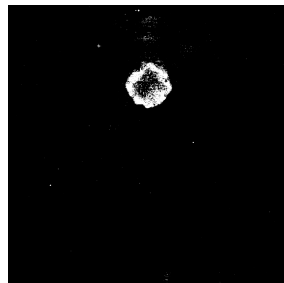
As you can see above, there are some very strange images and there is no indication of any radiance, even in the right side for the medium configuration image without the crazy throughput noise, there seems to be no indication of radiance being applied. This requires a more intricate look at both the integration and the slice visualizations

## Slice Visualizations

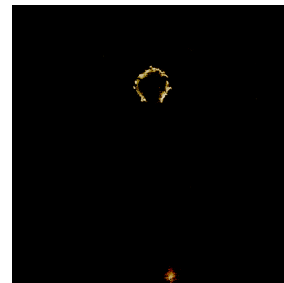
Below are some z-slice visualizations for the large configuration model, with the left being the radiance and right being transmittance (red channel only to bypass the cyan being shown, see [cyan\\_background.pdf](#) for more detail)



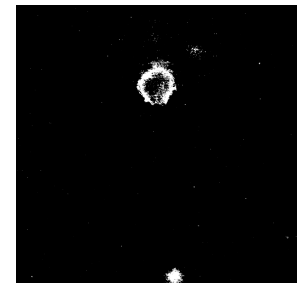
medium\_9 radiance



medium\_9\_T



medium\_10 radiance



medium\_10\_T

Though they look like they would be similar shapes and that the fire is surrounding it, looking at a specific UV coordinate in `large_9` gives the bottom most emissive position to be (508, 375) but the corresponding coordinate in the transmittance gives a white pixel value, indicating full opacity (white in transmittance is fully opaque). This means that its predicting it to be fully opaque and thus not showing the radiance. The reason why is because the network calculates the inherent

throughput of the volume but in the path tracing it calculates a cumulative throughput. During delta tracking, emission is also added regardless of what kind of scattering event occurs and is scaled up by the majorant.

```
// Add emission, if present. Always do this and scale
// by sigma_a/sigma_maj rather than only doing it
// (without scaling) at absorption events.
if (w.depth < maxDepth && mp.Le)
{
    Float pr = sigma_maj[0] * T_maj[0];
    SampledSpectrum r_e = r_u * sigma_maj * T_maj / pr;

    // Update _L_ for medium emission
    if (r_e)
        L += beta * mp.sigma_a * T_maj * mp.Le /
            (pr * r_e.Average());
}
```

And thus, looking at the alignment of the radiance and transmittance, it shows that there is radiance when there is a transmittance of 0 (fully opaque) and thus when calculating the volume's contribution to lighting would effectively put it at 0

## Integration Limitation

Following on from the slice mismatch given above, the naive integration and how it treats scattering is also likely causing this to fail. In the original, a scattering event during delta tracking would cause medium scattering work item to be enqueue which would trace shadow rays to the nearest light to calculate indirect lighting/shadowing or if there is no path to a light would cause an indirect ray to be enqueued, causing further medium sampling to occur. However, due to the nature of the I-NGP network trained and the fact it attempts to calculate full lighting contribution for the whole volume and the fact it attempts to be deterministic without doing delta tracking, it does not enqueue any medium scatter work items and instead acts as if the ray “skips through” the volume without doing any more sampling/scattering. Due to this, even if I were to set the beta/transmittance to be  $> 0$  if non-zero radiance was predicted, it would just cause it to look as if it was transparent and go straight through the volume.

A potential fix for this is to just enqueue another ray sample if radiance is detected to enqueue more medium sampling work in the next kernel invocation, thus ensuring the emission can contribute to final pixel values. However, this would deviate too much from the original plan of a “simple” integration and the original algorithm. Furthermore, there were other issues with radiance with different or larger architectures, see [network\\_fidelity\\_and\\_radiance\\_issues.pdf](#), that putting more effort into this may only realize results for a single configuration. Thus, it was decided to shift the evaluation to just looking at the transmittance to see how well the network learnt the actual shape of the volumes themselves, as opposed to both radiance and transmittance. This also leads to a slightly simpler visual quality evaluation. This then gives some focus for future work to require looking at a better integration or network architecture to ensure radiance can be properly rendered as it was evident from the slice visualization that some network configurations can indeed learn radiance, but it was mainly an oversight on the integration stage that lead to this not being realized in the rendering.